databricks

Continuous Integration (CI)

LECTURE

Version Control with Git Overview

© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Git version control is important because it provides a structured way to manage, track, and collaborate on code changes in software projects. Basically, it is an essential tool for modern software development and DevOps.

Complications with Version Control

Organizational Challenges

- Silos form over time that isolate development and team operations.
- Independent development leads to lower quality of software development due to lack of centralized version control (duplicate code, inconsistent standards, code reviewing, etc.)
- Unstable versioning – it can become difficult to track, revert, and audit changes .
- Without proper version control, managing frequent updates increases risk.
- Branching, merging, and CI/CD integration can become difficult, which diminishes the ability to scale development.

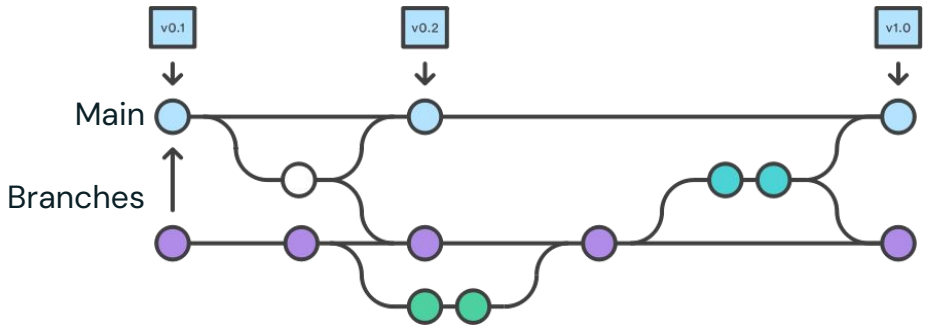
© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

[Read complications with version control bullets or read the below summary]

A lack of centralized version control leads to development silos, resulting in duplicate code, inconsistent standards, and lower software quality. Unstable versioning makes tracking, reverting, and auditing changes difficult, increasing risks when managing frequent updates. Additionally, branching, merging, and CI/CD integration become challenging, ultimately limiting scalability.

Secure code changes through branching

- Complementary concept to CI/CD → Enables effective CI
- Version control changes and run through quality control before merging to main branch and deploying
- Example: Gitflow



Source: Openclipart.org

© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

This is an example of a version control and code management over time with Gitflow, which is a common branching strategy that organizes branches for features, releases, and hotfixes.

- Here you will see different versions of a feature (v0.1, v0.2, and v1.0).
- Starting with the first version v0.1 on the main branch, branching occurs to the feature branch where testing and be properly performed in isolation.

Now that we know how complicated version control can be and we've seen an example of what securing code changes looks like, let's take a look at how Git can be used with Databricks.

Overview of Git with Databricks

Definitions

Git is a free and open-source software framework designed to track changes in source code during software development

© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Git Benefits: Version control enables tracking code changes, facilitating rollback and collaboration. Branching and merging allow multiple developers to work in parallel and integrate changes efficiently. A distributed job ensures each developer has a full local repository, enhancing flexibility and reliability. Git is optimized for high-performance handling of large projects, and its security features use cryptographic integrity checks to prevent data corruption.

Git Tools & Services

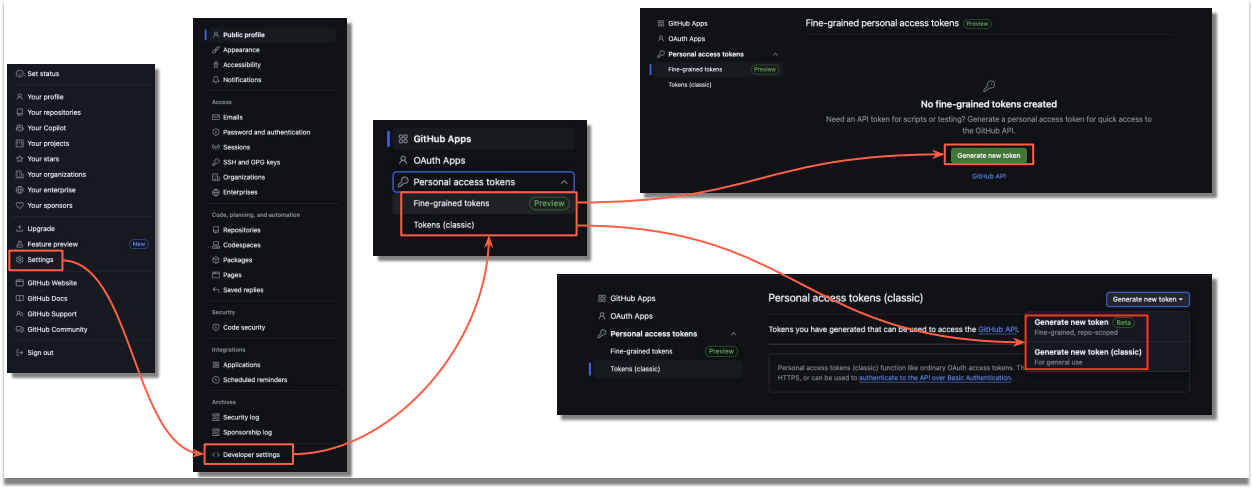
1. **GitHub, GitLab, Bitbucket, Azure DevOps** – Cloud-based repositories with CI/CD, issue tracking, and team collaboration.
2. **Git CLI & GUI Clients (e.g., SourceTree, GitKraken, VS Code Git Integration)** – Provides different interfaces for managing repositories.
3. **CI/CD Integration** – Automated testing and deployment pipelines.
4. **Code Review & Collaboration** – Features like pull requests and merge approvals streamline teamwork.
5. **Security & Access Control** – Role-based permissions and audit logs enhance repository security.

1. **Visual Git Client -** Databricks provides a user-friendly interface for common Git operations, which we will discuss about on the next slide.

1. developing code inside Databricks notebooks
2. **CI/CD Capabilities -** The repos REST API enables integration of data and AI projects into CI/CD pipelines, allowing users to automate Git Lakeflow Jobs

Generating GitHub Personal Access Token(PAT)

Authentication

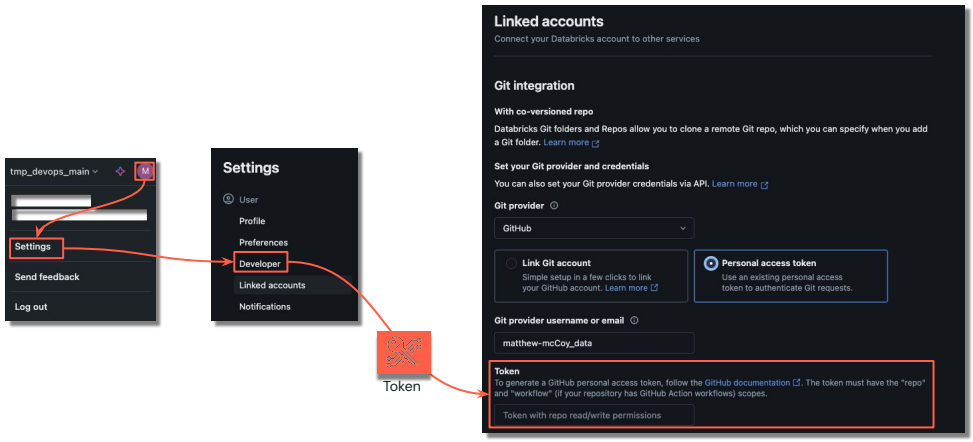


© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

To generate a Personal Access Token (PAT) in GitHub, go to **Settings** → **Developer settings**, choose either fine-grained or classic tokens, and click **Generate new token**. Configure the required repository permissions, copy the token, and use it in Databricks for integration.

Connecting to Databricks with GitHub PAT

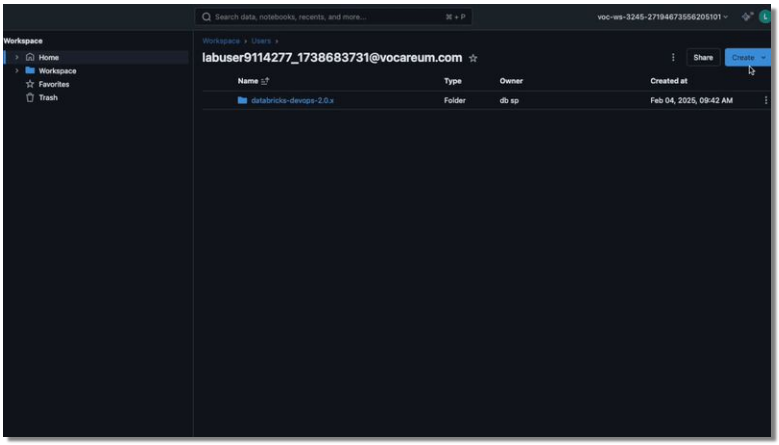
Seamless Integration



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Setting up a Personal Access Token (PAT) and connecting to your repositories is simple—click your user icon, go to **Settings** → **Developer settings** → **Personal access tokens**, then copy the token into the provided text box. Once saved, your account is linked and ready to use with your repository.

Databricks Git Folders



1. Clone remote repositories
2. Commit and push changes using UI
3. Pull branch updates
4. Branch Management
5. Visual validation when committing

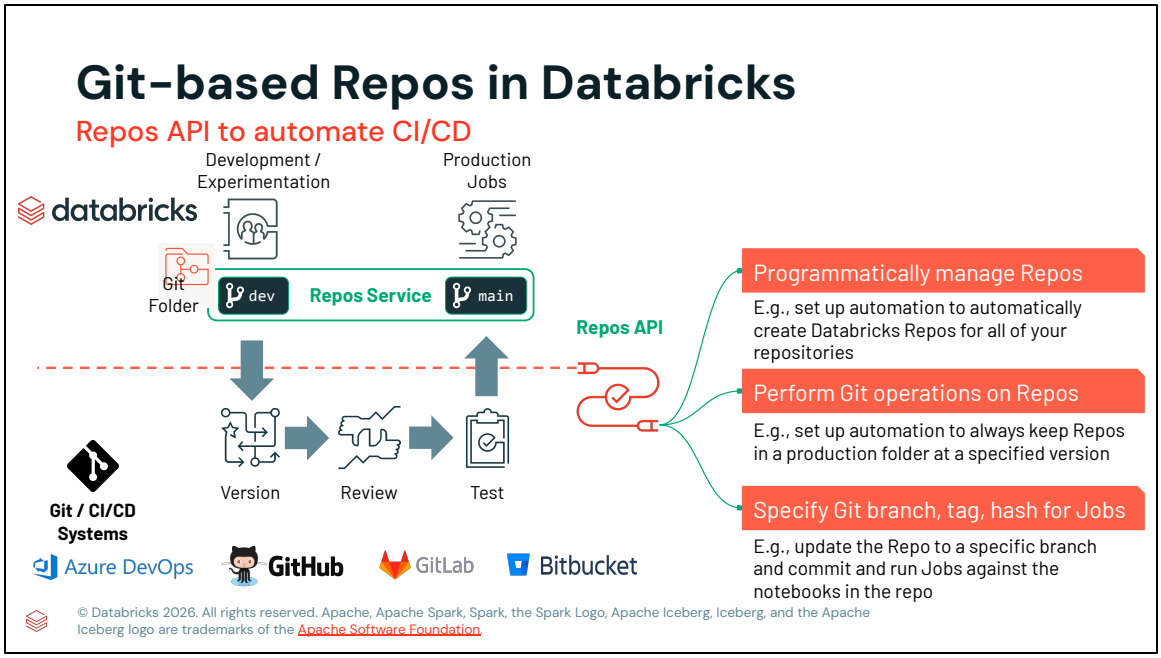
© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Databricks Git folders streamline development Lakeflow Jobs by tightly integrating version control systems into the Databricks ecosystem, making it easier to manage code collaboratively while adhering to best practices.

Use-cases include:

- Collaborative development of machine learning models and ETL pipelines.
- Source-controlling SQL queries for analytics Lakeflow Jobs.
- Automating deployments through CI/CD pipelines.

In the GIF provided here, you can see how easy it is to paste your URL from GitHub into the creation window for your Git folder.



1. Aha! Feature: [DB-3749 W2.0: Projects API private preview with tag and commit based checkouts | Aha!]

2. Description of the features:

- The Repos API provides programmatic access to git-based Re[ps that are part of the Workspace 2.0 effort. With the API, customers can integrate Databricks Repos with their CI/CD workflow. They can programmatically create/update/delete Repos, perform git operations, and specify Git versions when running Jobs based on notebooks in Repos.

3. Value of the feature (aka what databricks was before this feature, and what this feature will do for databricks)

- Right now, many customers have built workarounds using the Databricks CLI to pull notebooks from Databricks, check them into Git, pull them from Git, and push them back to Databricks. This is not a very robust solution. With Repos and the Repos API we provide a native

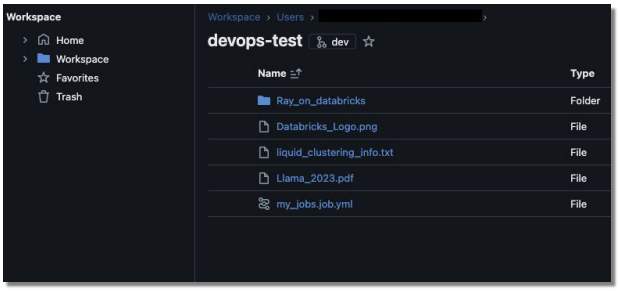
- feature to pull code from a Git repository, check updates back into Git, and programmatically update this Repos using the Repos API

4. Associated summary slide bullet points: Repos API for CI/CD integration

5. Cloud: All

6. Deployment (MT, ST, Azure): GA

Arbitrary files support in Repos



.py

- Portability of code
- Library files - use Python/R files as packages

.txt
.yml

- Environment specification portability
- Build packages from the same repo

.csv

- Small data ease of use
- Relative imports

...

- Whatever you can do with files "just works"

© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Arbitrary files are supported in repos, enabling portability of code for library files like Python and R. This makes it possible to build packages from the same repository, use relative imports, and structure your project in a way that best suits your needs.

© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Thank you for completing this lesson and continuing your journey to develop your skills with us.